Chapitre 0 — Avant tout | Partie B — Le temps | leçon 0.14.1 sur 153 | difficulté 1 / 10

0.14.1. Deux choses, deux rythmes — de base, ailleurs

Le 0.14 en colonne 10 : la lente et la rapide, au milieu de la ligne.

1. Ce qu'il faut comprendre

C'est le 0.14, avec un seul changement : la **colonne** des deux `poser` : 10 au lieu de 0.

Les chronomètres ne changent pas : seules les cases où l'on affiche les deux lettres ont bougé.

C'est la version de base : une seule chose, à la deuxième place. Le 0.14.2 met les deux ensemble.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police
> - `#include <poser>` → 0.1.3. La fonction poser() — poser une tuile

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **if (si)** — Teste une condition : si elle est vraie, on fait le bloc qui suit.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **incrémenter (++)** — « x++ » ou « x += 1 » : x augmente de 1. « x-- » : x diminue de 1.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **poser()** — Met une tuile dans le décor, sur une case de la grille de 20 × 18 (32 × 32 en tout, avec la partie cachée).
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`//` `————————————————————————————————————————————————————————`	Un commentaire : une phrase pour celui qui lit (« —— »). La console l'ignore.
`// LES VARIABLES : écrites AVANT main(), elles existent pendant`	Un commentaire : une phrase pour celui qui lit (« LES VARIABLES : écrites AVANT main(), elles existent pendant »). La console l'ignore.
`// tout le programme et gardent leur valeur d'un tour à l'autre.`	Un commentaire : une phrase pour celui qui lit (« tout le programme et gardent leur valeur d'un tour à l'autre. »). La console l'ignore.
`// uint8_t = un nombre entier de 0 à 255.`	Un commentaire : une phrase pour celui qui lit (« uint8_t = un nombre entier de 0 à 255. »). La console l'ignore.
`//` `————————————————————————————————————————————————————————`	Un commentaire : une phrase pour celui qui lit (« —— »). La console l'ignore.
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`#include <poser>  // pose une tuile sur une case du fond`	Demande à la console « poser » (ce que c'est : pose une tuile sur une case du fond). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « pose une tuile sur une case du fond ».)
`uint8_t lente = 0;  // QUELLE lettre montre la lente : 0 = A, 1 = B … 25 = Z`	Une variable : une boîte appelée lente, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « QUELLE lettre montre la lente : 0 = A, 1 = B … 25 = Z ».)
`uint8_t rapide = 0;  // QUELLE lettre montre la rapide : 0 = A, 1 = B … 25 = Z`	Une variable : une boîte appelée rapide, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « QUELLE lettre montre la rapide : 0 = A, 1 = B … 25 = Z ».)
`uint8_t imagesLente = 0;  // le CHRONOMÈTRE de la lente :`	Une variable : une boîte appelée imagesLente, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « le CHRONOMÈTRE de la lente : ».)
`// combien d'images depuis son dernier changement`	Un commentaire : une phrase pour celui qui lit (« combien d'images depuis son dernier changement »). La console l'ignore.
`uint8_t imagesRapide = 0;  // le CHRONOMÈTRE de la rapide, séparé de l'autre`	Une variable : une boîte appelée imagesRapide, qui retient un nombre de 0 à 255, et qui commence à 0. (Le commentaire le dit : « le CHRONOMÈTRE de la rapide, séparé de l'autre ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`// La boucle de jeu : elle ne s'arrête JAMAIS.`	Un commentaire : une phrase pour celui qui lit (« La boucle de jeu : elle ne s'arrête JAMAIS. »). La console l'ignore.
`// Chaque tour dure exactement une image, soit 1/60e de seconde.`	Un commentaire : une phrase pour celui qui lit (« Chaque tour dure exactement une image, soit 1/60e de seconde. »). La console l'ignore.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();  // attend l'image suivante (≈ 16,7 ms) :`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « attend l'image suivante (≈ 16,7 ms) : ».)

`// c'est ce qui cadence la boucle à 60 tours par seconde`	Un commentaire : une phrase pour celui qui lit (« c'est ce qui cadence la boucle à 60 tours par seconde »). La console l'ignore.
`// À chaque tour, UNE image est passée : les deux chronomètres`	Un commentaire : une phrase pour celui qui lit (« À chaque tour, UNE image est passée : les deux chronomètres »). La console l'ignore.
`// avancent d'un cran, en même temps, chacun de son côté.`	Un commentaire : une phrase pour celui qui lit (« avancent d'un cran, en même temps, chacun de son côté. »). La console l'ignore.
`imagesLente++;  // ++ veut dire « ajoute 1 »`	imagesLente augmente de 1. (Le commentaire le dit : « ++ veut dire « ajoute 1 » ».)
`imagesRapide++;`	imagesRapide augmente de 1.
`// ── LA LENTE : elle change toutes les 60 images = toutes les secondes ──`	Un commentaire : une phrase pour celui qui lit (« ── LA LENTE : elle change toutes les 60 images = toutes les secondes ── »). La console l'ignore.
`if (imagesLente == 60) {  // == veut dire « est égal à ? »`	Une décision : si imagesLente est égal à 60, on fait ce qui est entre les accolades. Sinon, on passe. (Le commentaire le dit : « == veut dire « est égal à ? » ».)
`// (un seul = voudrait dire « range »)`	Un commentaire : une phrase pour celui qui lit (« (un seul = voudrait dire « range ») »). La console l'ignore.
`imagesLente = 0;  // son chronomètre repart de zéro`	imagesLente prend une nouvelle valeur : 0. (Le commentaire le dit : « son chronomètre repart de zéro ».)
`lente++;  // elle passe à la lettre suivante`	lente augmente de 1. (Le commentaire le dit : « elle passe à la lettre suivante ».)
`if (lente == sizeof(ALPHABET)) lente = 0; // sizeof(ALPHABET) = 26 :`	Une décision : si lente est égal à sizeof(ALPHABET) alors on fait « lente = 0 ». Sinon, on passe. (Le commentaire le dit : « sizeof(ALPHABET) = 26 : ».)
`// après Z (25), 26 n'existe pas :`	Un commentaire : une phrase pour celui qui lit (« après Z (25), 26 n'existe pas : »). La console l'ignore.
`// retour à A (0)`	Un commentaire : une phrase pour celui qui lit (« retour à A (0) »). La console l'ignore.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`// ── LA RAPIDE : elle change toutes les 15 images = 4 fois par seconde ──`	Un commentaire : une phrase pour celui qui lit (« ── LA RAPIDE : elle change toutes les 15 images = 4 fois par seconde ── »). La console l'ignore.
`// (60 images ÷ 15 = 4 changements par seconde)`	Un commentaire : une phrase pour celui qui lit (« (60 images ÷ 15 = 4 changements par seconde) »). La console l'ignore.
`if (imagesRapide == 15) {`	Une décision : si imagesRapide est égal à 15, on fait ce qui est entre les accolades. Sinon, on passe.
`imagesRapide = 0;  // son chronomètre repart de zéro`	imagesRapide prend une nouvelle valeur : 0. (Le commentaire le dit : « son chronomètre repart de zéro ».)
`rapide++;  // elle passe à la lettre suivante`	rapide augmente de 1. (Le commentaire le dit : « elle passe à la lettre suivante ».)
`if (rapide == sizeof(ALPHABET)) rapide = 0;  // après Z, retour à A`	Une décision : si rapide est égal à sizeof(ALPHABET) alors on fait « rapide = 0 ». Sinon, on passe. (Le commentaire le dit : « après Z, retour à A ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

`// ── L'AFFICHAGE : à CHAQUE tour, on repose les deux lettres ──`	Un commentaire : une phrase pour celui qui lit (« ── L'AFFICHAGE : à CHAQUE tour, on repose les deux lettres ── »). La console l'ignore.
`// Si rien n'a changé, on repose la même lettre au même endroit : ça ne se voit pas.`	Un commentaire : une phrase pour celui qui lit (« Si rien n'a changé, on repose la même lettre au même endroit : ça ne se voit pas. »). La console l'ignore.
`// Si l'indice a changé, la nouvelle lettre écrase l'ancienne (comme au 0.3).`	Un commentaire : une phrase pour celui qui lit (« Si l'indice a changé, la nouvelle lettre écrase l'ancienne (comme au 0.3). »). La console l'ignore.
`poser(10, 0, ALPHABET[lente]);  // la lente : colonne 0 → 10`	Appel de la fonction poser de la console, avec les arguments 10, 0, ALPHABET[lente]. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans le fond (colonne et ligne en clair). (Le commentaire le dit : « la lente : colonne 0 → 10 ».)
`poser(10, 2, ALPHABET[rapide]);  // la rapide : colonne 0 → 10`	Appel de la fonction poser de la console, avec les arguments 10, 2, ALPHABET[rapide]. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans le fond (colonne et ligne en clair). (Le commentaire le dit : « la rapide : colonne 0 → 10 ».)
`// (la ligne 1 reste vide pour les séparer)`	Un commentaire : une phrase pour celui qui lit (« (la ligne 1 reste vide pour les séparer) »). La console l'ignore.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **940 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        61 o  ← la console, à l'allumage  — la mise en route de la console
  main()           91 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte,
EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard,
EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  poser() × 2    24 o sur place, dans main()
  image() × 1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ————————————————————————————————————————————————————
// LES VARIABLES : écrites AVANT main(), elles existent pendant
// tout le programme et gardent leur valeur d'un tour à l'autre.
// uint8_t = un nombre entier de 0 à 255.
// ————————————————————————————————————————————————————

#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <poser>      // pose une tuile sur une case du fond
```

2. UNE VARIABLE — LENTE

```
uint8_t lente = 0;          // QUELLE lettre montre la lente : 0 = A, 1 = B … 25 = Z
```

3. UNE VARIABLE — RAPIDE

```
uint8_t rapide = 0;         // QUELLE lettre montre la rapide : 0 = A, 1 = B … 25 = Z
```

4. UNE VARIABLE — IMAGESLENTE

```
uint8_t imagesLente = 0;    // le CHRONOMÈTRE de la lente :
                            // combien d'images depuis son dernier changement
```

5. UNE VARIABLE — IMAGESRAPIDE

```
uint8_t imagesRapide = 0;   // le CHRONOMÈTRE de la rapide, séparé de l'autre
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {

  // La boucle de jeu : elle ne s'arrête JAMAIS.
  // Chaque tour dure exactement une image, soit 1/60e de seconde.
  while (true) {

    image();               // attend l'image suivante (≈ 16,7 ms) :
                           // c'est ce qui cadence la boucle à 60 tours par seconde

    // À chaque tour, UNE image est passée : les deux chronomètres
    // avancent d'un cran, en même temps, chacun de son côté.
    imagesLente++;         // ++ veut dire « ajoute 1 »
    imagesRapide++;

    // ── LA LENTE : elle change toutes les 60 images = toutes les secondes ──
    if (imagesLente == 60) {                    // == veut dire « est égal à ? »
                                                // (un seul = voudrait dire « range »)
      imagesLente = 0;                          // son chronomètre repart de zéro
      lente++;                                  // elle passe à la lettre suivante
      if (lente == sizeof(ALPHABET)) lente = 0; // sizeof(ALPHABET) = 26 :
                                                // après Z (25), 26 n'existe pas :
                                                // retour à A (0)
    }

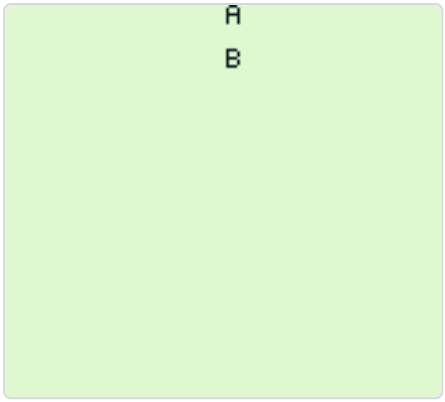
    // ── LA RAPIDE : elle change toutes les 15 images = 4 fois par seconde ──
    // (60 images ÷ 15 = 4 changements par seconde)
    if (imagesRapide == 15) {
      imagesRapide = 0;                         // son chronomètre repart de zéro
      rapide++;                                 // elle passe à la lettre suivante
      if (rapide == sizeof(ALPHABET)) rapide = 0;  // après Z, retour à A
    }

    // ── L'AFFICHAGE : à CHAQUE tour, on repose les deux lettres ──
    // Si rien n'a changé, on repose la même lettre au même endroit : ça ne se voit pas.
    // Si l'indice a changé, la nouvelle lettre écrase l'ancienne (comme au 0.3).
    poser(10, 0, ALPHABET[lente]);    // la lente : colonne 0 → 10
    poser(10, 2, ALPHABET[rapide]);   // la rapide : colonne 0 → 10
                                      // (la ligne 1 reste vide pour les séparer)
  }
}
```

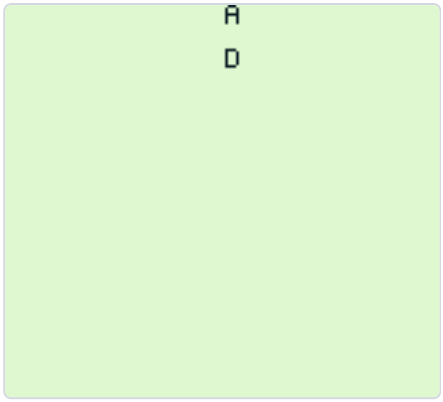
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

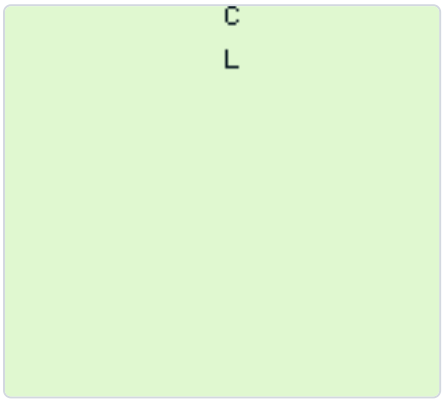
la toute première image — la console vient de démarrer — image 1



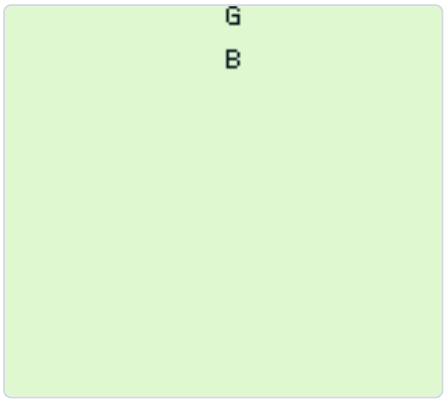
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

> La lente et la rapide, en colonne 10.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ colonne 10 : la lente (ligne 0) et la rapide (ligne 2) changent

---

Ensuite : **0.14.2. Deux choses, deux rythmes — doublé, deux positions** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++